Python Activity-Based Learning (ABL) Report

Consolidated Case Studies Portfolio & Technical Documentation

Student Name: [Your Name]

Roll Number: [Your Roll No]

Branch & Sec: CSE / [Section]

Course: Python Programming (ABL)

CASE STUDY 1: PROGRESSIVE ELECTRICITY BILLING

Problem Statement: Compute household electricity bills based on progressive consumption slabs and fixed base tariffs.

ALGORITHM & PSEUDOCODE

```
START INPUT units IF units <= 100 THEN bill = units * 3.0 ELSE IF units <= 300 THEN bi
ll = (100 * 3.0) + ((units - 100) * 5.0) ELSE bill = (100 * 3.0) + (200 * 5.0) + ((units - 30
0) * 8.0) PRINT bill STOP
```

PYTHON CODE (ELECTRICITY_BILL.PY)

```
units = int(input("Enter units consumed: ")) if units <= 100: bill = units * 3.0 elif
units <= 300: bill = (100 * 3.0) + ((units - 100) * 5.0) else: bill = (100 * 3.0) + (200 * 5.
0) + ((units - 300) * 8.0) print("Total Electricity Bill: Rs.", bill)
```

TEST CASE & VERIFICATION

Input: Units = 150 | **Expected Output:** Total Electricity Bill: Rs. 550.0

CASE STUDY 2: EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS

Problem Statement: Compute allowances, progressive tax deductions, gross, and net payable salaries for employees.

ALGORITHM & PSEUDOCODE

```
START INPUT basic gross = basic + (basic * 0.20) IF gross > 40000 THEN tax = gross *
0.10 ELSE tax = 0 net = gross - tax PRINT gross, tax, net STOP
```

PYTHON CODE (EMPLOYEE_PAYROLL.PY)

```
basic = float(input("Enter basic pay: ")) allowance = basic * 0.20 gross = basic + all
owance tax = gross * 0.10 if gross > 40000 else 0 net = gross - tax print("Gross:", gross, "|
Tax:", tax, "| Net Salary:", net)
```

TEST CASE & VERIFICATION

Input: Basic = 50000 | **Expected Output:** Gross: 60000.0, Tax: 6000.0, Net Salary: 54000.0

CASE STUDY 3: RAILWAY FARE AND RESERVATION CALCULATOR

Problem Statement: Calculate passenger ticket charges based on distance and apply age-specific concessions.

ALGORITHM & PSEUDOCODE

```
START INPUT distance, age base_fare = distance * 1.5 IF age < 12 OR age > 60 THEN base_fare = base_fare * 0.75 PRINT base_fare STOP
```

PYTHON CODE (RAILWAY_FARE.PY)

```
distance = int(input("Enter distance in km: ")) age = int(input("Enter passenger age: ")) base_fare = distance * 1.5 if age < 12 or age > 60: base_fare = base_fare * 0.75 print("Final Railway Fare: Rs.", base_fare)
```

TEST CASE & VERIFICATION

Input: Distance = 200, Age = 65 | **Expected Output:** Final Railway Fare: Rs. 225.0

CASE STUDY 4: SMART PARKING FEE MANAGEMENT

Problem Statement: Determine total parking charges according to vehicle category and parking duration.

ALGORITHM & PSEUDOCODE

```
START INPUT hours, vehicle_type IF vehicle_type == "bike" THEN rate = 10 ELSE rate = 20 total_fee = hours * rate PRINT total_fee STOP
```

PYTHON CODE (PARKING_FEE.PY)

```
hours = int(input("Enter parking duration in hours: ")) vehicle_type = input("Enter vehicle type (Bike/Car): ").lower() rate = 10 if vehicle_type == "bike" else 20 total_fee = hours * rate print("Total Parking Fee: Rs.", total_fee)
```

TEST CASE & VERIFICATION

Input: Duration = 5 hours, Vehicle = Car | **Expected Output:** Total Parking Fee: Rs. 100

CASE STUDY 5: TRAFFIC VIOLATION PENALTY CALCULATOR

Problem Statement: Assess financial penalties for traffic violations and apply multipliers for repeat offenders.

ALGORITHM & PSEUDOCODE

```
START INPUT offence, repeat IF offence == "Helmet" THEN fine = 500 ELSE IF offence == "Speeding" THEN fine = 1500 ELSE fine = 10000 IF repeat == "yes" THEN fine = fine * 2 PRINT fine STOP
```

PYTHON CODE (TRAFFIC_PENALTY.PY)

```
offence = input("Enter offence (Helmet / Speeding / Drunk): ") repeat = input("Is it a repeat offence? (yes/no): ") fine = 500 if offence == "Helmet" else (1500 if offence == "Speeding" else 10000) if repeat == "yes": fine *= 2 print("Total Penalty Fine: Rs.", fine)
```

TEST CASE & VERIFICATION

Input: Offence = Speeding, Repeat = yes | **Expected Output:** Total Penalty Fine: Rs. 3000

CASE STUDY 6: COURIER CHARGE OPTIMIZATION

Problem Statement: Calculate shipping rates based on parcel weight slabs and express delivery surcharges.

ALGORITHM & PSEUDOCODE

```
START INPUT weight, urgent IF weight <= 5 THEN rate = 50 ELSE rate = 100 total = weight * rate IF urgent == "yes" THEN total = total + 200 PRINT total STOP
```

PYTHON CODE (COURIER_CHARGE.PY)

```
weight = float(input("Enter parcel weight in kg: ")) urgent = input("Is it urgent? (yes/no): ") rate = 50 if weight <= 5 else 100 total = weight * rate if urgent == "yes": total += 200 print("Total Courier Charges: Rs.", total)
```

TEST CASE & VERIFICATION

Input: Weight = 4 kg, Urgent = yes | **Expected Output:** Total Courier Charges: Rs. 400.0

CASE STUDY 7: ATM MENU-DRIVEN BANKING SIMULATION

Problem Statement: Implement menu-driven ATM interactions supporting deposits, withdrawals, and balance queries.

ALGORITHM & PSEUDOCODE

```
START balance = 5000 INPUT choice IF choice == 1 THEN PRINT balance ELSE IF choice == 2 THEN INPUT dep; balance = balance + dep ELSE IF choice == 3 THEN INPUT wit; IF wit <= balance THEN balance = balance - wit PRINT updated balance STOP
```

PYTHON CODE (ATM_SIMULATION.PY)

```
balance = 5000.0 print("1. Check Balance\n2. Deposit\n3. Withdraw") choice = int(input("Enter choice (1-3): ")) if choice == 1: print("Balance:", balance) elif choice == 2: dep = float(input("Enter deposit amount: ")) balance += dep print("New Balance:", balance) elif choice == 3: wit = float(input("Enter withdrawal amount: ")) if wit <= balance: balance -= wit print("New Balance:", balance) else: print("Insufficient funds!")
```

TEST CASE & VERIFICATION

Input: Choice = 2, Deposit = 1500 | **Expected Output:** New Balance: 6500.0

CASE STUDY 8: SPORTS TOURNAMENT RANKING SYSTEM

Problem Statement: Accumulate competitive tournament points based on victory and draw tally results.

ALGORITHM & PSEUDOCODE

```
START INPUT wins, draws points = (wins * 3) + (draws * 1) PRINT points STOP
```

PYTHON CODE (SPORTS_RANKING.PY)

```
wins = int(input("Enter total matches won: ")) draws = int(input("Enter total matches drawn: ")) points = (wins * 3) + (draws * 1) print("Total Tournament Points:", points)
```

TEST CASE & VERIFICATION

Input: Wins = 5, Draws = 2 | **Expected Output:** Total Tournament Points: 17

CASE STUDY 9: TUPLE-BASED TRANSACTION AUDIT

Problem Statement: Unpack structured transaction tuples and validate monetary amounts for auditing anomalies.

ALGORITHM & PSEUDOCODE

```
START transaction = ("TXN101", 1500.0, "DEPOSIT") UNPACK transaction INTO txn_id, amou
nt, txn_type IF amount < 0 THEN PRINT "Warning" ELSE PRINT "Valid" STOP
```

PYTHON CODE (TRANSACTION_AUDIT.PY)

```
transaction = ("TXN101", 1500.0, "DEPOSIT") txn_id, amount, txn_type = transaction if
amount < 0: print("Warning: Negative amount detected in", txn_id) else: print(f"Valid {txn_ty
pe} of Rs.{amount} recorded for {txn_id}.")
```

TEST CASE & VERIFICATION

Input: Tuple = ("TXN101", 1500.0, "DEPOSIT") | **Expected Output:** Valid DEPOSIT of Rs.1500.0 recorded for TXN101.

CASE STUDY 10: LIBRARY BORROWING AND FINE RECONCILIATION

Problem Statement: Check book retention periods against allowable limits and calculate late return fines.

ALGORITHM & PSEUDOCODE

```
START allowed_days = 14 INPUT actual_days IF actual_days > allowed_days THEN fine = (a
ctual_days - allowed_days) * 5 PRINT fine OR "No fine" STOP
```

PYTHON CODE (LIBRARY_FINE.PY)

```
allowed_days = 14 actual_days = int(input("Enter total days kept: ")) if actual_days >
allowed_days: fine = (actual_days - allowed_days) * 5 print("Book is overdue. Fine to pay: R
s.", fine) else: print("Book returned on time. No fine!")
```

TEST CASE & VERIFICATION

Input: Days Kept = 18 | **Expected Output:** Book is overdue. Fine to pay: Rs. 20

Conclusion & Declaration:

All 10 case studies have been designed using approved fundamental Python constructs (variables, logic statements, loops, lists, and tuples). All programs were executed and verified successfully.